

Name: ReadMatrix

Inputs:

lines: multiline string where each line is consists of the same number of comma-separated substrings. Each substring represents a real numerical value.

Outputs:

a matrix M such that $M(i,j)$ is the i-th value on the j-th line in lines

1. **rows** $\leftarrow$ an empty sequence
2. for each **line** in **lines** do
 - a. **row** $\leftarrow$ an empty sequence
 - b. for each comma-separated substring **num_string** in **line** do
 - i. **num_from_string** $\leftarrow$ the real number represented by **num_string**
 - ii. append **num_from_string** at the end of the sequence **row**
 - c. append the sequence **row** at the end of **rows**
3. **matrix** $\leftarrow$ a **matrix** for which each ith row where i ranges from 0 to **num_rows**, is given by the sequence **rows_i**, for all elements of **rows**
4. return matrix

Name: GeneratePredictions

Inputs:

I: a m x 1 matrix of the weights of the linear model

V: a n xm matrix representing the explanatory variables

1. Return **VI**

Name: PredictionError

Inputs:

weights: a m x 1 matrix of the weights of the linear model

inputs: a n x m matrix representing the explanatory variables

result: a n x 1 matrix representing the actual values of the measured variables

Outputs: a float representing the MSE of the generated data from the actual data

1. **total** $\leftarrow$ 0
2. **predictions** $\leftarrow$ *GeneratePredictions(weights, inputs)*
3. For each **row** in 0,1,..., n-1, do:
 - a. **total** $\leftarrow$ **total** + (**result**_{row, 0} - **predictions**_{row, 0})²
4. Return **total** / n

Least Squares Derivation

$$\frac{1}{n}(2w^T X^T x - 2y^T x) = 0$$

$$2w^T X^T x = 2y^T x$$

$$w^T X^T x = y^T x$$

$$(w^T X^T x)^T = (y^T x)^T$$

$$x^T X w = x^T y$$

$$w = (X^T X)^{-1} X^T y$$

Link: https://py3.codeskulptor.org/#user311_3iqHO7BCKY_17.py

Data:

Least-Squares:

Training error: 93.73996861983848

Test error: 105.0855290618747

LASSO (lambda=1000)

Training error: 122.6008523128453

Test error: 102.8834996003878

LASSO (lambda=50000)

Training error: 139.1888580752672

Test error: 92.85986437777377

LASSO (lambda=100000)

Training error: 141.4045710247012

Test error: 95.56592869241656

Discussion:

1. Expression is true for this computation since the matrices used multiply to a 1×1 matrix, which is always equal to its own transpose (e.g. $c = c^T$).
2. As lambda increases, weights become less complex, which increases the error on the training dataset. This occurs due to the LASSO estimate sacrificing accuracy, as it is now minimizing both values instead of just the MSE when $\lambda = 0$. When the weights become less complex, they approach zero, and some are forced to zero.
3. The least-squares model had the lowest MSE on the training data, but performed a lot worse on the test set. The medium-lambda LASSO model achieves the lowest MSE on the test data. This result that the least-squares model is overfitting, resulting in a low error on the training data. Alternatively the LASSO model ($\lambda = 1000$) has a higher training error but achieves the lowest test error, implying that it is good at generalization. Lasso is less complex and thus more generalized and not specific for only the training data.
4. Yes. One can use generated fitted weights using the LASSO algorithm to help determine the statistics that are not important. LASSO identifies the statistics that are not important by reducing their weights to zero. Any statistic whose weight is zero can be deemed not important since its contribution to the final output is insignificant.